

Unidad III: Funciones map y manejo de errores en iteraciones

Isaac Alexander Morales Arévalo

Contents

3. Ejercicios	1
Ejercicio 1	1
Ejercicio 2	2
Ejercicio 3	2
Ejercicio 4	2
Ejercicio 5	2
Ejercicio 6	2
Ejercicio 7	3
Ejercicio 8	4
Ejercicio 9	4
Ejercicio 10	4

```
library(purrr)
library(dplyr)
library(tibble)
```

3. Ejercicios

Ejercicio 1

Usa `map_dbl()` para calcular la media de cada columna de `mtcars`.

```
library(purrr)
map_dbl(mtcars, mean)
```

```
##      mpg      cyl    disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am     gear     carb
##  0.437500  0.406250  3.687500  2.812500
```

Ejercicio 2

Usa `map_dbl()` para calcular la desviación estándar de cada columna de `mtcars`.

```
library(purrr)
map_dbl(mtcars, sd)
```

```
##      mpg      cyl      disp      hp      drat      wt
## 6.0269481 1.7859216 123.9386938 68.5628685 0.5346787 0.9784574
##      qsec      vs      am      gear      carb
## 1.7869432 0.5040161 0.4989909 0.7378041 1.6152000
```

Ejercicio 3

Usa `map_chr()` para obtener la clase de cada columna de `iris`.

```
map_chr(iris, class)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## "numeric"    "numeric"    "numeric"    "numeric"    "factor"
```

Ejercicio 4

Usa `map_lgl()` para determinar qué columnas de `iris` son numéricas.

```
map_lgl(iris, is.numeric)
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## TRUE         TRUE         TRUE         TRUE         FALSE
```

Ejercicio 5

Usa `map_int()` para calcular el número de valores únicos en cada columna de `iris`.

```
library(purrr)
map_int(iris, ~ length(unique(.x)))
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 35           23           43           22           3
```

Ejercicio 6

Usa `map_dbl()` para calcular el rango de cada columna de `mtcars`, definido como:

$$\max(x) - \min(x).$$

```
map_dbl(mtcars, function(x) {
  max(x) - min(x)
})
```

```
##      mpg      cyl    disp      hp      drat      wt      qsec      vs      am      gear
## 23.500  4.000 400.900 283.000  2.170  3.911  8.400  1.000  1.000  2.000
##      carb
##      7.000
```

Ejercicio 7

Crea una lista con los valores 1, 10, "a" y 100. Usa `safely()` para aplicar `log()` sin que el proceso se detenga.

```
x <- list(1, 10, "a", 100)

log_seguro <- safely(log)

resultado <- map(x, log_seguro)

resultado
```

```
## [[1]]
## [[1]]$result
## [1] 0
##
## [[1]]$error
## NULL
##
##
## [[2]]
## [[2]]$result
## [1] 2.302585
##
## [[2]]$error
## NULL
##
##
## [[3]]
## [[3]]$result
## NULL
##
## [[3]]$error
## <simpleError in .Primitive("log")(x, base): Argumento no numérico para una función matemática>
##
##
## [[4]]
## [[4]]$result
## [1] 4.60517
##
## [[4]]$error
## NULL
```

Ejercicio 8

Usa el resultado del ejercicio anterior para identificar cuáles elementos produjeron error.

```
map_lgl(resultado, ~ !is.null(.x$error))
```

```
## [1] FALSE FALSE  TRUE FALSE
```

Ejercicio 9

Usa `possibly()` para aplicar `log()` a la lista `x <- list(1, 10, "a", 100)`. Cuando ocurra un error, debe devolver `NA`.

```
log_posible <- possibly(log, otherwise = NA)
```

```
map(x, log_posible)
```

```
## [[1]]
## [1] 0
##
## [[2]]
## [1] 2.302585
##
## [[3]]
## [1] NA
##
## [[4]]
## [1] 4.60517
```

Ejercicio 10

Usa `quietly()` para aplicar `log()` a los valores 1, -1, 10 y -5. Luego extrae las advertencias generadas.

```
valores <- list(1, -1, 10, -5)
```

```
log_silencioso <- quietly(log)
```

```
resultado_quiet <- map(valores, log_silencioso)
```

```
resultado_quiet
```

```
## [[1]]
## [[1]]$result
## [1] 0
##
## [[1]]$output
## [1] ""
##
## [[1]]$warnings
## character(0)
```

```

##
## [[1]]$messages
## character(0)
##
##
## [[2]]
## [[2]]$result
## [1] NaN
##
## [[2]]$output
## [1] ""
##
## [[2]]$warnings
## [1] "Se han producido NaNs"
##
## [[2]]$messages
## character(0)
##
##
## [[3]]
## [[3]]$result
## [1] 2.302585
##
## [[3]]$output
## [1] ""
##
## [[3]]$warnings
## character(0)
##
## [[3]]$messages
## character(0)
##
##
## [[4]]
## [[4]]$result
## [1] NaN
##
## [[4]]$output
## [1] ""
##
## [[4]]$warnings
## [1] "Se han producido NaNs"
##
## [[4]]$messages
## character(0)

```